

项目级 VibeCoding

Stitch · AI Studio · 灵码 · Claude Code · Expo Go · MCP

"Vibe Coding：用自然语言驱动，从原型到上线的全栈开发新范式"

模块	核心工具
界面原型设计	Google Stitch
Web 前端生成	Google AI Studio
Web 后端生成	Claude Code
移动端 App 开发	通义灵码 + Expo (React Native)
移动端真机调试	Expo Go
Web 端自动调试	MCP chrome-devtools

什么是 VibeCoding?

"You fully give in to the vibes, embrace exponentials, and forget that the code even exists."
——Andrej Karpathy, 2025

核心思想：不再逐行手写代码，而是用自然语言描述意图，让 AI 完成实现。

传统开发	VibeCoding
需要精通语言语法	只需清晰描述需求
手动搭建项目结构	AI 自动生成脚手架
前后端分别学习	统一用对话驱动
耗时数天搭原型	数小时跑通全栈

本讲工具链总览

三步走，完成一个完整的 Web 应用：

Step 1 —▶ Google Stitch

用 AI 快速设计界面原型（视觉稿 + 可交互）

Step 2 —▶ Google AI Studio

将原型描述转化为真实前端代码（React / Vue）

Step 3 —▶ Claude Code

生成后端 API，对接前端，完成全栈集成

三个工具各司其职——设计、前端、后端——串联成一条完整的 AI 辅助开发流水线

PART 1

| 用 Stitch 设计界面原型

从想法到可交互原型，只需一句话

什么是 Google Stitch?

Google 推出的 AI 驱动界面原型工具:

- 输入自然语言描述, 自动生成 App / Web 界面原型
- 支持移动端与桌面端双端预览
- 可导出为组件代码 (HTML / React)
- 与 Google 设计规范 (Material Design) 深度整合

访问地址: stitch.withgoogle.com

适合场景:

- 快速验证产品想法 (PoC)
- 与团队对齐视觉方案, 再交给 AI 生成代码
- 非设计师也能产出专业原型

Stitch 核心操作流程

- ① 打开 Stitch → 新建项目
- ② 在提示框中描述你的应用：
"设计一个校园二手书交易 App，
包含首页列表、商品详情、发布页面三个页面"
- ③ Stitch 自动生成界面原型（约 10–30 秒）
- ④ 在画布上点击各组件，可继续对话修改：
"把导航栏改成底部 Tab 栏，颜色使用蓝色系"
- ⑤ 满意后 → 导出原型截图 / 组件代码

关键技巧：提示词中明确说明页面数量、核心功能、用户场景，生成质量更高

Stitch 提示词示例

场景：为软件工程课程项目设计一个"任务协作平台"

提示词（第一轮）：

设计一个团队任务管理 Web 应用。包含：① 任务看板页（Kanban 风格，待办 / 进行中 / 已完成）；② 任务详情侧边栏（标题、描述、负责人、截止日期）；③ 顶部导航（Logo、搜索框、用户头像）。使用现代简洁风格。

提示词（精修）：

把任务卡片改为带颜色标签的卡片，优先级用红 / 黄 / 绿色区分。
侧边栏增加评论区。

Stitch 与传统 UI 工具对比

维度	Figma	Stitch
学习成本	中等（需学习组件系统）	低（自然语言驱动）
生成速度	手动拖拽，较慢	AI 自动生成，秒级
代码导出	插件辅助，质量参差	原生支持，格式标准
协作功能	强（多人实时编辑）	基础
适用阶段	精细设计稿	快速原型验证

VibeCoding 场景下，Stitch 用于快速对齐需求，Figma 用于最终交付物——两者互补

PART 2

| 用 AI Studio 生成前端代码

从原型描述到可运行的 React 项目

什么是 Google AI Studio?

aistudio.google.com —— 基于 Gemini 模型的 Web 端 AI 开发平台:

- 支持长上下文 (100万+ tokens), 可上传整个原型截图 + 需求文档
- 内置代码生成、调试、解释功能
- 可直接在浏览器中预览 HTML/JS 代码
- 通过 API Key 接入 Gemini 模型, 也可用于项目中调用 AI 能力

VibeCoding 中的定位:

- 上传 Stitch 导出的原型截图
- 描述前端技术栈 (React + Tailwind / Vue + Element Plus)
- 让 Gemini 生成完整的前端组件代码

前端项目初始化

Step 1: 创建 React 项目

```
# 使用 Vite 初始化前端项目 (推荐)
npm create vite@latest my-app-frontend -- --template react
cd my-app-frontend
npm install

# 安装常用依赖
npm install axios react-router-dom tailwindcss
```

Step 2: 在 AI Studio 中输入提示词

我用 Stitch 设计了一个任务看板应用的原型（截图已上传）。
请用 React + Tailwind CSS 实现首页看板组件。
数据先用 mock 数据，API 接口后端地址为 `http://localhost:3000/api`。

AI Studio 生成前端代码

在 AI Studio 对话框中输入上述提示词后，Gemini 会：

1. **分析原型截图** → 理解布局结构与组件层次
2. **生成 React 组件** → `KanbanBoard.jsx`、`TaskCard.jsx`、`Sidebar.jsx`
3. **自动处理样式** → 使用 Tailwind utility class
4. **预留 API 接口** → `axios.get('/api/tasks')` 占位

将代码粘贴到项目后继续追问：

生成的卡片组件缺少拖拽功能，请添加 react-beautiful-dnd 实现列间拖拽。

AI Studio 的长上下文优势：可以一次性粘贴多个文件的代码，让 AI 理解整体结构后再修改

前端生成：关键提示词技巧

目标	提示词模板
生成新页面	"基于已有的 <code>Layout.jsx</code> 结构，生成【页面名】页面组件"
修改样式	"把卡片的背景色改为紫色系，与南大配色 <code>#4D0099</code> 保持一致"
接入真实 API	"将 mock 数据替换为调用 <code>GET /api/tasks</code> 的真实请求，处理 loading 和 error 状态"
添加交互	"点击卡片弹出侧边栏，使用 Framer Motion 添加滑入动画"
代码审查	"检查此组件是否有内存泄漏或 React key 警告问题"

PART 3

| 用 Claude Code 生成后端

从自然语言到可运行的 Node.js API

什么是 Claude Code?

Anthropic 推出的 **终端 AI 编程 Agent**:

- 在本地终端运行, 直接读写项目文件
- 理解整个代码库上下文, 不只是单个文件
- 自主执行命令 (`npm install`、`git`、运行测试)
- 支持多轮对话, 持续完善代码

```
# 安装 Claude Code
```

```
npm install -g @anthropic-ai/claude-code
```

```
# 在项目目录中启动
```

```
cd my-app-backend
```

```
claude
```

VibeCoding 中的定位: 负责生成和维护后端 API, 与前端对接

后端项目初始化

在项目根目录下，对 Claude Code 说：

为我生成这个 App 的后端代码。
后端目录名为 `my-app-backend`，与 `my-app-frontend` 同级。
采用 Node.js，使用 Express 框架，数据库用 SQLite（开发简单）。

Claude Code 将自动执行：

```
mkdir my-app-backend  
cd my-app-backend && npm init -y  
npm install express cors dotenv better-sqlite3  
npm install --save-dev nodemon
```

并生成 `src/index.js`、`src/routes/`、`src/db/` 等完整项目结构

Claude Code 生成后端 API

继续对话：

实现任务管理的 CRUD API：

- `GET /api/tasks` 获取全部任务列表
- `POST /api/tasks` 新建任务
- `PATCH /api/tasks/:id` 更新任务状态
- `DELETE /api/tasks/:id` 删除任务

每个任务包含：id、title、status (todo/doing/done)、priority、assignee、dueDate

Claude Code 会自动：

- 创建 SQLite 数据库表
- 编写路由处理函数
- 添加基本的输入校验
- 生成测试用的 seed 数据

后端代码示例

Claude Code 生成的核心路由结构:

```
// src/routes/tasks.js
const express = require('express');
const router  = express.Router();
const db      = require('../db');

// 获取所有任务 (支持按 status 过滤)
router.get('/', (req, res) => {
  const { status } = req.query;
  const tasks = status
    ? db.prepare('SELECT * FROM tasks WHERE status = ?').all(status)
    : db.prepare('SELECT * FROM tasks ORDER BY created_at DESC').all();
  res.json(tasks);
});

// 新建任务
router.post('/', (req, res) => {
  const { title, priority, assignee, dueDate } = req.body;
  const result = db.prepare(
    'INSERT INTO tasks (title, status, priority, assignee, due_date) VALUES (?, ?, ?, ?, ?)'
  ).run(title, 'todo', priority, assignee, dueDate);
  res.status(201).json({ id: result.lastInsertRowid });
});

module.exports = router;
```

接入 AI 能力：调用 API Key

让后端具备 AI 功能（以 Gemini API 为例）：

在后端新增一个接口 `POST /api/ai/suggest`，
接收任务标题，调用 Google Gemini API，
返回 AI 建议的任务分解步骤。
API Key 从环境变量 `GEMINI_API_KEY` 读取。

```
// Claude Code 生成的 AI 接口
const { GoogleGenerativeAI } = require('@google/generative-ai');
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

router.post('/suggest', async (req, res) => {
  const { title } = req.body;
  const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });
  const result = await model.generateContent(
    `将以下任务分解为 3-5 个子步骤，用中文回答：${title}`
  );
  res.json({ steps: result.response.text() });
});
```

PART 4

通义灵码开发移动端 App

用灵码从零生成 iOS / Android 原生应用

通义灵码移动端开发概览

通义灵码（VS Code 插件 / Lingma IDE）同样支持移动端开发流程：

- 与前面的 AI Studio + Claude Code workflow 并行的另一条路线
- 适合团队已在使用阿里云生态（DashScope API / 通义系列模型）
- 全程在 VS Code 内完成：原型描述 → 生成代码 → 运行调试

移动端技术栈选择：

技术栈	特点	适合场景
Expo (React Native)	JS/TS, 热重载, Expo Go 预览	快速原型, 跨平台
Flutter	Dart, 高性能, UI 统一	对 UI 精度要求高
原生 Swift / Kotlin	最佳性能, 学习成本高	生产级 App

VibeCoding 推荐：Expo (React Native) + 通义灵码，上手最快，与 Web 前端知识复用度最高

初始化 Expo 移动端项目

在 VS Code 终端运行以下命令，初始化 App 前端代码库：

```
# 使用 expo 初始化一个 app 的前端代码库  
npx create-expo-app@latest
```

命令运行过程中，系统会提示为前端项目命名，命名为：`demo-app-frontend`。运行完成后，生成代码库。

```
# 进入该项目的代码库  
cd demo-app-frontend  
  
# 运行  
npm start
```

在灵码中生成移动端界面

打开 VS Code，激活通义灵码插件，在对话框中输入：

把 home 页面改成一个 AI 对话页面，在其中，可以像在使用通义 app 一样，与 AI 进行文字对话

灵码将自动：

- ① 查看项目目录结构（已查看目录）
- ② 查看 app 目录，了解页面结构
- ③ 查看 tabs 目录，了解页面结构
- ④ 查看首页（index.tsx）当前代码，了解需要修改的内容
- ⑤ 生成新的 home 页面代码（AI 对话界面）

关键技巧：灵码会先自动读取项目结构，再生成代码——无需手动粘贴文件内容，直接描述需求即可

灵码生成后端服务

继续在灵码对话框中输入：

现在，为我生成这个 app 的后端代码。后端的代码文件名称为 demo-app-backend，与 demo-app-frontend 同级。后端采用 nodejs，使用 express 框架

灵码将自动执行以下命令：

```
mkdir demo-app-backend
cd demo-app-backend && npm init -y
cd demo-app-backend && npm install express cors dotenv
cd demo-app-backend && npm install --save-dev nodemon
```

灵码生成了完整的 nodejs 后端代码库 `demo-app-backend`，与 `demo-app-frontend` 一个是 app 的后端，另一个是前端。

设置前后端通信（灵码版）

继续对话：

使用 RTK query 作为前端与后端通信的方式

灵码将自动安装 redux 等依赖，并实现用 RTK query 做前后端的通信方案。

```
# 灵码自动执行
cd demo-app-frontend
npm install @reduxjs/toolkit react-redux
```

然后自动生成：

```
src/
  store/
    index.ts      ← Redux store 配置
  api/
    tasksApi.ts   ← RTK Query API 定义 (GET / POST / PATCH / DELETE)
  components/
    ChatScreen.tsx ← AI 对话界面组件
```

调用 API Key（接入通义大模型）

下面，设置 nodejs 后端连接到 qwen 的 api 服务。此处以 qwen-plus 模型做示例，需要准备百炼的 `DASHSCOPE_API_KEY`。

在灵码对话框中输入：

下面，我们先完善后端服务，让后端与 qwen api 连接。这里是百炼提供的 nodejs 连接到 qwen api 的方式。`DASHSCOPE_API_KEY` 将配置在 nodejs 后端的 .env 文件中。

```
import OpenAI from "openai";

const openai = new OpenAI({
  // 若没有配置环境变量，请用百炼API Key替换为: apiKey: "sk-xxx"
  apiKey: process.env.DASHSCOPE_API_KEY,
  baseURL: "https://dashscope.aliyuncs.com/compatible-mode/v1"
});

async function main() {
  const completion = await openai.chat.completions.create({
    model: "qwen-plus", // 可按需更换模型名称
    messages: [
      { role: "system", content: "You are a helpful assistant." },
      { role: "user", content: "你是谁? " }
    ],
  });
}
```

Expo Go App 模拟器调试

Expo Go 是移动端开发的核心调试利器：

安装 Expo Go：在 iPhone / Android 手机上，从 App Store / Google Play 安装 Expo Go。

扫码预览：

```
cd demo-app-frontend
npm start
# 终端输出二维码 → 手机 Expo Go 扫码 → 实时预览
```

热重载机制：灵码修改代码 → 文件保存 → Expo Go 自动刷新，无需重新编译

模拟器调试流程：

调试场景	操作
界面布局错误	Shake 手机 → 开启 Inspector，点击元素查看样式
网络请求失败	Expo Dev Tools → Network 面板查看请求详情
JS 报错	红屏报错 → 直接复制给灵码，让 AI 修复
性能问题	Expo Dev Tools → Performance Monitor

Expo Go 调试：与灵码协作

最佳实践：红屏报错 → 直接截图粘贴给灵码

【截图 Expo Go 红屏错误信息】

上面是我的 Expo Go 报错，请分析原因并修复。

灵码会自动：

1. 读取相关源文件
2. 定位错误原因
3. 修改代码并说明改动

常见错误对应提示：

```
"Unable to resolve module '@components/ChatScreen'"
```

→ 告诉灵码："路径别名 @ 没有配置，请修复 tsconfig.json 中的 paths"

```
"Network request failed"
```

→ 告诉灵码："后端地址 localhost 在真机上不可用，
请替换为本机 IP 地址，并在灵码帮我找到正确的 IP"

注意：手机调试时后端地址不能用 localhost，需改为电脑的局域网 IP（如 192.168.1.x:3000）

PART 5

| MCP: chrome-devtools 自动调试 Web 端

让 AI 直接操控浏览器，自动定位并修复前端问题

什么是 MCP?

MCP (Model Context Protocol): AI 与工具之间的标准通信协议

- 由 Anthropic 提出, 已成为 AI 工具集成的事实标准
- 允许 AI (Claude Code / 灵码 等) 直接调用外部工具
- 每个 MCP Server 赋予 AI 一种新能力, 按需开启

AI Agent (Claude Code)

↕ MCP 协议

chrome-devtools	→	读取浏览器状态
github	→	操作 GitHub
vercel	→	一键部署
mcp-server-sqlite	→	查询数据库
mcp-server-chart	→	生成图表

过去: AI 看不到浏览器里发生了什么, 只能根据你的文字描述来猜测问题。现在: AI 可以直接读取控制台报错、网络请求、DOM 结构。

Claude Code 中的 MCP 服务器列表

在 Claude Code 的设置面板中，可以看到已配置的 MCP Server 及其连接状态：

图标	MCP Server	状态	用途
●	vercel	连接失败	自动部署到 Vercel 平台
●	github	已连接	读写 Issue、PR、仓库文件
●	chrome-devtools	连接失败	读取浏览器 Console / Network / DOM
●	mastergo-magic-mcp	已连接	读取 MasterGo 设计稿，生成代码
●	mcp-server-chart	已连接	生成 ECharts / Chart.js 图表
●	mcp-server-sqlite	已连接	直接查询 SQLite 数据库

● 表示 MCP Server 进程未启动或配置有误； ● 表示已正常连接

chrome-devtools 连接失败 通常原因：Chrome 未以调试端口启动，见下页解决方法。

配置 MCP chrome-devtools

Step 1: 在 Claude Code 配置文件中声明

```
// ~/.claude/settings.json (全局) 或 .claude/settings.json (项目)
{
  "mcpServers": {
    "chrome-devtools": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-puppeteer"],
      "env": {
        "PUPPETEER_LAUNCH_OPTIONS": "{\"headless\": false}"
      }
    }
  }
}
```

Step 2: 以远程调试模式启动 Chrome

```
# macOS - 让 Chrome 开放调试端口 9222
/Applications/Google\ Chrome.app/Contents/MacOS/Google\ Chrome \
  --remote-debugging-port=9222 --no-first-run --no-default-browser-check

# Windows
"C:\Program Files\Google\Chrome\Application\chrome.exe" --remote-debugging-port=9222
```

其他 MCP Server 在 VibeCoding 中的应用

除 chrome-devtools 外，列表中其他 MCP Server 同样能加速开发：

github

直接对 Claude Code 说：

"帮我查看仓库最近 5 个 Issue，找出与登录相关的 bug"

"把当前修改提交并创建 PR，标题写明修复了什么"

mcp-server-sqlite

"查询 tasks 表中所有 status='todo' 的记录，统计各 priority 数量"

"帮我检查数据库中是否存在重复的用户邮箱"

mcp-server-chart

"根据数据库中的任务完成数据，生成一个按周统计的折线图"

vercel（配置后）

"帮我把前端项目部署到 Vercel，项目名叫 my-app"

MCP chrome-devtools 自动调试流程

启动后，直接对 Claude Code 说：

帮我调试当前页面，任务看板的卡片拖拽功能失效，拖拽时没有任何反应，也没有报错提示。

Claude Code 通过 MCP 自动执行：

- ① 连接 Chrome DevTools (CDP 协议)
- ② 读取 Console 面板 → 检查是否有隐藏的 JS 错误
- ③ 读取 Network 面板 → 检查拖拽时是否有 API 请求发出
- ④ 检查 DOM 结构 → 确认 react-beautiful-dnd 的 Draggable 容器是否正确渲染
- ⑤ 在浏览器中注入调试代码 → 监听 dragstart / dragend 事件
- ⑥ 分析结果 → 定位问题根因 → 修改源代码 → 通知刷新页面

全程无需手动打开 DevTools!

MCP chrome-devtools: 实战示例

场景：登录后页面空白，不知道原因

对 Claude Code 说：

登录成功后跳转到首页，但页面空白。
请用 chrome-devtools 帮我查明原因。

Claude Code 通过 MCP 自动完成：

```
// 1. 读取 Console 错误
// → 发现: Uncaught TypeError: Cannot read properties of undefined (reading 'map')
// → 位置: TaskList.jsx:42

// 2. 读取 Network 请求
// → 发现: GET /api/tasks 返回 200, 但 response body 为 { "data": null }
//    而前端代码直接 tasks.map(...), 没有处理 null 情况

// 3. 自动修复建议:
const tasks = data?.tasks ?? []; // 添加空值保护
```

MCP 的价值：AI 从"盲人摸象"变为"亲眼观察"，调试效率提升 5—10x

MCP chrome-devtools 常用调试场景

场景	对 Claude Code 说
控制台报错	"读取当前页面的所有 Console 错误，逐一分析并修复"
网络请求失败	"查看 Network 面板，找出所有 4xx/5xx 请求，分析原因"
页面样式异常	"检查 <code>.task-card</code> 的实际计算样式，找出为什么宽度变成 0"
性能问题	"Profile 首页加载过程，找出耗时超过 100ms 的操作"
内存泄漏	"拍摄堆内存快照，比较前后差异，找出未释放的对象"
接口联调	"拦截 POST /api/tasks 请求，打印 request body 和 response"

PART 6

前后端集成与运行

打通全链路，看到最终效果

设置前后端通信

前端使用 **Axios** 对接后端 API:

告诉 AI Studio (或 Claude Code):

将前端所有 mock 数据替换为真实 API 调用。

后端地址: `http://localhost:3000`,

请统一封装 axios 实例, 处理错误和 loading 状态。

```
// src/api/index.js (AI Studio 生成)
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3000',
  timeout: 10000,
});

export const getTasks = () => api.get('/api/tasks');
export const createTask = (data) => api.post('/api/tasks', data);
export const updateTask = (id, data) => api.patch(`/api/tasks/${id}`, data);
export const deleteTask = (id) => api.delete(`/api/tasks/${id}`);
```

启动与联调

```
# 终端 1: 启动后端
cd my-app-backend
npm run dev
# → 监听 http://localhost:3000

# 终端 2: 启动前端
cd my-app-frontend
npm run dev
# → 访问 http://localhost:5173
```

常见问题与 Claude Code 修复：

问题	对 Claude Code 说
跨域报错 (CORS)	"后端开启 CORS, 允许 localhost:5173 访问"
接口 404	"检查路由注册是否正确, 打印所有已注册路由"
数据格式不匹配	"前端收到的字段是 <code>due_date</code> , 改为 <code>dueDate</code> 驼峰命名"
SQLite 并发写入	"开启 WAL 模式解决并发写入问题"

完整 VibeCoding workflow 回顾

【需求】

- ↓ 用自然语言描述产品功能

【Stitch 原型】

- ↓ 生成可视化界面原型，与团队对齐页面结构

【AI Studio → Web 前端】

- ↓ 原型截图 + 技术栈描述
- ↓ 生成 React 组件 / 路由 / 样式

【通义灵码 → 移动端 App】

- ↓ VS Code 内对话生成 Expo 代码
- ↓ RTK Query 前后端通信

【Claude Code / 灵码 → 后端】

- ↓ 描述 API 需求与数据模型
- ↓ 自动生成 Express 路由、数据库
- ↓ 接入 AI API (Gemini / 通义)

【调试】

- ↓ Web 端: MCP chrome-devtools 自动读取控制台/网络/DOM
- ↓ 移动端: Expo Go 扫码热重载, 红屏报错直接给灵码修复

【集成联调】

- ↓ 前后端通信、CORS、数据格式对齐

【上线】

- 前端部署 Vercel, 后端部署 Railway / Render

VibeCoding 的软件工程思考

VibeCoding 改变了什么？

- 编程的门槛从"语法精通"降低为"需求表达能力"
- 软件工程师的价值从"写代码"转向"定义问题 + 审查 AI 输出"
- 原型验证速度提升 10x，但**最终质量仍取决于人的判断**

VibeCoding 没有改变什么？

- 需求分析、系统设计、架构决策——仍是人的责任
- 安全性、性能、可维护性——AI 生成的代码需要人工审查
- 测试与部署——自动化工具可以辅助，但不能省略

警惕：AI 生成的代码可能包含安全漏洞（SQL 注入、未校验输入）。上线前必须做安全审查！

本次项目作业

目标：用 VibeCoding 工具链完成一个小型全栈应用

阶段	要求
原型	用 Stitch 设计至少 3 个页面，截图存档
前端	用 AI Studio 生成 React 项目，或用灵码生成 Expo App
后端	用 Claude Code / 灵码 生成 REST API，至少 4 个端点
调试	Web 端配置 MCP chrome-devtools，移动端用 Expo Go 联调
集成	前后端联通，数据能正常读写
报告	记录每步使用的提示词，分析 AI 的局限性

提交方式：GitHub 仓库 + 提示词日志文档

评分重点不在于功能多少，而在于你如何引导 AI、如何处理 AI 的错误输出

六讲总结：AI4Coding + VibeCoding 的完整图景

08-01 AI 辅助编程入门

└─ 工具生态 · 六大能力 · 提示词四要素

08-02 开发工具链与 Git 工作流

└─ DevOps · Git 四区域 · Vercel 自动部署

08-03 AI 编程进阶与 Agent 时代

└─ 项目级技巧 · 代码质量 · 软件工程的未来演进

06 前端开发基础

└─ Web 演进 · Vue.js · Node.js

07 Java Web 开发

└─ Spring Boot · MyBatis · Spring Security

08 项目级 VibeCoding

└─ Stitch · AI Studio · 灵码 · Claude Code · Expo Go · MCP

核心洞察：AI 工具改变的是速度和交互方式，不变的是需求分析、架构设计与工程纪律

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

第 6 讲 项目级 VibeCoding

本讲核心收获

- └─ Stitch: 自然语言描述 → 可视化界面原型, 快速对齐产品需求
- └─ AI Studio + Claude Code: Web 前后端全链路 AI 生成与联调
- └─ 通义灵码 + Expo Go: 移动端 App 开发与真机热重载调试
- └─ MCP chrome-devtools: AI 直读浏览器状态, 自动定位前端问题

VibeCoding 改变了开发的速度, 不变的是工程思维的价值